

QN]

Evaluate divide and conquer matrix multiplication by implementing recursive splitting of matrices. Record execution time for different matrix sizes. Analyze how problem division reduces complexity. Interpret results and justify the efficiency of recursive approaches.

Code:

```
import java.util.Random;
```

```
public class DivideAndConquerMatrixMultiplication {
```

```
    static int[][] add(int[][] A, int[][] B) {
```

```
        int n = A.length;
```

```
        int[][] C = new int[n][n];
```

```
        for (int i = 0; i < n; i++)
```

```
            for (int j = 0; j < n; j++)
```

```
                C[i][j] = A[i][j] + B[i][j];
```

```
        return C;
```

```
    }
```

```
    static int[][] multiply(int[][] A, int[][] B) {
```

```
        int n = A.length;
```

```
        int[][] C = new int[n][n];
```

```
        if (n == 1) {
```

```
            C[0][0] = A[0][0] * B[0][0];
```

```
            return C;
```

```
        }
```

```
        int newSize = n / 2;
```

```
        int[][] A11 = new int[newSize][newSize];
```

```
        int[][] A12 = new int[newSize][newSize];
```

```
        int[][] A21 = new int[newSize][newSize];
```

```

int[][] A22 = new int[newSize][newSize];

int[][] B11 = new int[newSize][newSize];
int[][] B12 = new int[newSize][newSize];
int[][] B21 = new int[newSize][newSize];
int[][] B22 = new int[newSize][newSize];

for (int i = 0; i < newSize; i++) {
    for (int j = 0; j < newSize; j++) {
        A11[i][j] = A[i][j];
        A12[i][j] = A[i][j + newSize];
        A21[i][j] = A[i + newSize][j];
        A22[i][j] = A[i + newSize][j + newSize];

        B11[i][j] = B[i][j];
        B12[i][j] = B[i][j + newSize];
        B21[i][j] = B[i + newSize][j];
        B22[i][j] = B[i + newSize][j + newSize];
    }
}

```

```

int[][] C11 = add(multiply(A11, B11), multiply(A12, B12));
int[][] C12 = add(multiply(A11, B12), multiply(A12, B22));
int[][] C21 = add(multiply(A21, B11), multiply(A22, B21));
int[][] C22 = add(multiply(A21, B12), multiply(A22, B22));

```

```

for (int i = 0; i < newSize; i++) {
    for (int j = 0; j < newSize; j++) {
        C[i][j] = C11[i][j];
        C[i][j + newSize] = C12[i][j];
    }
}

```

```

        C[i + newSize][j] = C21[i][j];
        C[i + newSize][j + newSize] = C22[i][j];
    }
}

return C;
}

```

```

static int[][] generateMatrix(int n) {
    Random rand = new Random();
    int[][] matrix = new int[n][n];

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            matrix[i][j] = rand.nextInt(10);

    return matrix;
}

static void printMatrix(int[][] matrix) {
    for (int[] row : matrix) {
        for (int value : row)
            System.out.print(value + " ");
        System.out.println();
    }
}
}

```

```

public static void main(String[] args) {

    int[] sizes = {2, 4, 8, 16};
}

```

```
for (int n : sizes) {

    int[][] A = generateMatrix(n);
    int[][] B = generateMatrix(n);

    long start = System.nanoTime();

    int[][] C = multiply(A, B);

    long end = System.nanoTime();

    double time = (end - start) / 1_000_000.0;

    System.out.println("Matrix Size: " + n + " x " + n);
    System.out.printf("Execution Time: %.4f ms%n", time);

    if (n == 2) {
        System.out.println("\nMatrix A:");
        printMatrix(A);

        System.out.println("\nMatrix B:");
        printMatrix(B);

        System.out.println("\nResult Matrix:");
        printMatrix(C);
    }

    System.out.println("-----");
```

```
    }  
  }  
}
```

Output]

Matrix Size: 2 x 2

Execution Time: 0.0122 ms

Matrix A:

```
6 4  
3 1
```

Matrix B:

```
4 5  
7 6
```

Result Matrix:

```
44 54  
19 21
```

Matrix Size: 4 x 4

Execution Time: 0.0576 ms

Matrix Size: 8 x 8

Execution Time: 0.4531 ms

Matrix Size: 16 x 16

Execution Time: 2.5530 ms